

واجهات برمجة التطبيقات و FastAPI

ملخصات علوم الحاسب، الجزء 6 من 6

1. مفهوم API وأسلوب REST

واجهة برمجة التطبيقات (API) هي وسيلة تسمح لبرنامج بالتواصل مع برنامج آخر وفق قواعد معروفة. في الويب يرسل العميل (Client)، مثل المتصفح أو تطبيق الواجهة الأمامية، طلبا إلى الخادم (Server) عبر بروتوكول HTTP، ويرد الخادم بالنتيجة، وغالبا بصيغة JSON. أسلوب REST يعتمد على التعامل مع الموارد عبر عناوين URL وأفعال HTTP. الفعل GET لقراءة البيانات، و POST لإنشاء بيانات جديدة أو إرسال بيانات للمعالجة، و PUT لاستبدال مورد كامل، و PATCH لتعديل جزء منه، و DELETE للحذف. وتكون الخدمة عديمة الحالة (Stateless)، أي يحمل كل طلب المعلومات اللازمة لفهمه دون أن يعتمد الخادم على طلب سابق.

2. أكواد حالة HTTP

يرد الخادم دائما برمز حالة يوضح نتيجة الطلب:

- الرمز 200 (OK): نجح الطلب.
- الرمز 201 (Created): تم إنشاء مورد جديد بنجاح.
- الرمز 400 (Bad Request): الطلب غير صحيح.
- الرمز 404 (Not Found): المورد المطلوب غير موجود.
- الرمز 422 (Unprocessable Entity): البيانات المرسله لا تجتاز التحقق من الصحة. وهو الرمز الذي يرجعه FastAPI تلقائيا عندما يكون جسم الطلب غير مطابق للنموذج المعرف.
- الرمز 500 (Internal Server Error): حدث خطأ غير متوقع في الخادم.
- الرمز 503 (Service Unavailable): الخدمة غير متاحة مؤقتا، مثل عدم إمكانية الوصول إلى خدمة تعتمد عليها.

3. إطار العمل FastAPI

FastAPI إطار عمل حديث وسريع لبناء واجهات API بلغة بايثون، ويعتمد على مكتبة Starlette للتعامل مع الويب وعلى مكتبة Pydantic للتحقق من صحة البيانات. من مميزاته أنه ينشئ توثيقا تفاعليا تلقائيا: تجده على العنوان docs/ بواجهة Swagger UI وعلى العنوان redoc/. ويشغل التطبيق بخادم Uvicorn بالأمر التالي، حيث يعيد الخيار reload تشغيل الخادم عند تغيير الكود:

```
uvicorn app.main:app --reload
```

نعرف شكل البيانات باستخدام نماذج Pydantic. المثال التالي يعرف طلبا يحتوي على سؤال نصي، ومسارا من نوع POST يستقبله:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class QueryRequest(BaseModel):
    question: str
```

```
@app.post("/query")
def query(req: QueryRequest):
    return {"answer": "...", "sources": []}
```

إذا أرسل العميل جسم طلب لا يحتوي على الحقل question، يرد FastAPI برمز 422 دون أننفذ أي كود إضافي.

4. تحميل الموارد عند بدء التشغيل

بعض الموارد ثقيلة، مثل نموذج التضمين وقاعدة بيانات المتجهات. يجب تحميلها مرة واحدة عند بدء تشغيل التطبيق، وليس عند كل طلب، وإلا صار كل طلب بطيئاً جداً. في FastAPI نستخدم دالة lifespan لتنفيذ كود عند بدء التطبيق وعند إيقافه، ونحفظ الموارد المحملة في app.state لتستخدمها المسارات.

5. سياسة CORS ومتغيرات البيئة

سياسة CORS (Cross-Origin Resource Sharing) تعني مشاركة الموارد عبر المصادر المختلفة. المتصفح يمنع صفحة قادمة من مصدر معين من طلب بيانات من مصدر آخر، ما لم يسمح الخادم بذلك صراحة. لذلك نضيف CORSMiddleware إلى FastAPI ونحدد المصادر المسموح بها في allow_origins، مثل عنوان تطبيق الواجهة الأمامية.

لا يجوز كتابة العناوين والإعدادات السرية مباشرة داخل الكود، مثل عنوان الخادم http://localhost:8000. نضعها في ملف env. ونقرأها كمتغيرات بيئة باستخدام مكتبة python-dotenv أو مكتبة pydantic-settings. ولا نرفع ملف env. إلى GitHub أبداً، بل نرفع ملف env.example. يحتوي على أسماء المتغيرات وقيم افتراضية آمنة.

6. اختبار واجهة API

يوفر FastAPI الصنف TestClient الذي يسمح باختبار المسارات دون تشغيل خادم حقيقي، ويعمل مع أداة pytest. من الجيد كتابة اختبارات على الأقل: اختبار للحالة الناجحة يتوقع رمز 200، واختبار لمدخل غير صالح، مثل إرسال طلب بلا سؤال، يتوقع رمز 422.

```
from fastapi.testclient import TestClient
from app.main import app

def test_invalid_input():
    with TestClient(app) as client:
        r = client.post("/query", json={})
        assert r.status_code == 422
```

ولتجنب الاعتماد على خدمات خارجية أثناء الاختبار، يمكن استبدال خدمة النموذج اللغوي بنسخة وهمية تعيد إجابة ثابتة.